

CUBICAL NO 8

RAGAVARSHINI A (23AIB12)

SANJAY M (23AIB23)

SANJEEVIKUMAR M (23AIB27)

SINDHUJASREE R (23AIB33)

SIVASAKTHI R (23AIB36)

SOWMIYA G (23ECB38)

SOWMIYA P (23ECB39)

DAY 3 – External LED With Switch

DATE: 21-07-2025

DOMAIN: Embedded Systems and IoT

TOPIC: External LED With Switch

OBJECTIVE

To turn the LED off/on on the Arduino Uno board and show the status on the Serial monitor. LED will be in on state while switch is on otherwise LED should be off.

COMPONENTS REQUIRED

- 1.Arduino Uno Board
- 2.USB Cable
3. Arduino IDE (Software)

THEORY

- 1.LED anode (long leg) is connected to Arduino digital pin 8 (as output).
- 2.LED cathode (short leg) is connected to GND through a 220Ω resistor (current limiter).
- 3.One side of the push button is connected to Arduino digital pin 2 (as input).
Other side of the push button is connected to GND.
- 4.Arduino pin 2 is configured with INPUT_PULLUP to use an internal pull-up resistor. When the button is pressed, pin 2 reads LOW, and the Arduino turns LED ON.

PROCEDURE

1. LED anode (long leg) is connected to Arduino digital pin 8 (set as output).
- 2.LED cathode (short leg) is connected to GND through a 220Ω resistor.
- 3.One side of the push button is connected to Arduino digital pin 2 (set as input).
Other side of the button is connected to +5V (Vcc).

4. A 10k Ω pull-down resistor is connected between pin 2 and GND to ensure a LOW signal when the button is not pressed.

5. When the switch is pressed, pin 2 reads HIGH, and the LED turns ON; when released, it reads LOW and LED turns OFF.

ARDUINO CODE

1. LED ON while Button Pressed (Basic Control)

LED turns ON when button is pressed and OFF when released.

```
const int ledPin = 8;    // External LED connected to pin 8
const int buttonPin = 2; // Button connected to pin 2

void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP); // Using internal pull-up resistor
}

void loop() {
  int buttonState = digitalRead(buttonPin);

  if (buttonState == LOW) {      // Button pressed
    digitalWrite(ledPin, HIGH);  // LED ON
  } else {
    digitalWrite(ledPin, LOW);   // LED OFF
  }
}
```

2. LED Always ON Until Button Pressed

LED stays ON by default and turns OFF only while button is pressed.

```
const int ledPin = 8;

const int buttonPin = 2;


void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);
  digitalWrite(ledPin, HIGH);    // LED initially ON
}


void loop() {
  int buttonState = digitalRead(buttonPin);

  if (buttonState == LOW) {      // Button pressed
    digitalWrite(ledPin, LOW);    // Turn LED OFF
  } else {
    digitalWrite(ledPin, HIGH);   // Keep LED ON
  }
}
```

3. Toggle LED ON/OFF on Button Press

Press once = LED ON, press again = LED OFF (toggle effect).

```
const int ledPin = 8;

const int buttonPin = 2;
```

```
bool ledState = false;

bool lastButtonState = HIGH;

void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);
}

void loop() {
  bool currentButtonState = digitalRead(buttonPin);

  if (lastButtonState == HIGH && currentButtonState == LOW) {
    ledState = !ledState;          // Toggle LED state
    digitalWrite(ledPin, ledState);
    delay(200); // Simple debounce delay
  }

  lastButtonState = currentButtonState;
}
```

Result – External LED with Switch

- The external LED turned ON when the push button was pressed and OFF when released (basic control).
- When modified, the LED stayed always ON by default and turned OFF only when the button was pressed.
- In the toggle version, the LED switched between ON and OFF states with each button press — simulating real-world switch behavior.

Learning Outcome

1. Learned external LED control using `digitalWrite()` and hardware wiring.
2. Understood digital input handling using push buttons with `digitalRead()`.
 - Learned the use of internal pull-up resistors with `INPUT_PULLUP` to simplify switch wiring.
3. Gained experience in keeping output states constant or toggling based on input.
4. Explored variations of switch-based logic: momentary press, inverse logic, and toggle control.
5. Practiced debouncing logic and state tracking for real-time button interaction.